



暑期实践课程

——逻辑设计与FPGA基础

主讲人：林 婷
2026.06



基本语法与组合逻辑电路分析

主讲人：林 婷
2025.06

Verilog HDL语法与组合逻辑电路的设计



目录

- 1、 Verilog HDL基本语法
- 2、 组合逻辑电路的设计
- 3、 使用硬件描述语言来描述组合逻辑电路
- 4、 课堂练习



PART 01

Verilog HDL 基本语法

Verilog HDL基本语法

如何正确学习HDL代码？

- ❖ 参考学习语文/英语的经验
 - 阅读理解是学习语言的重中之重
 - 足够输入基础上的才可能有高质量的输出
- ❖ 深刻理解“描述”的含义
 - 不要以“执行软件”的思路来理解代码
 - 时刻思考代码与实际逻辑电路的对照关系
- ❖ 把握数字逻辑设计的根本
 - 代码是对数字逻辑的描述
 - 正确的逻辑设计是描述的前提
 - 没有足够的逻辑设计功底难以掌握语言本质

Verilog HDL基本语法

如何正确学习HDL代码？

❖ 学习阶段阅读HDL代码的意义

- 参考前人经验
- 熟悉常见电路描述方法

❖ 读懂HDL代码的前提

- 熟悉基本语法（后期学习中再继续深入）
- 熟悉常用逻辑电路（更加重要）

❖ 阅读HDL代码时的注意事项

- 注意从“描述”的角度理解
- 先总体后细节
- 注意和电路的对应关系
- 不要阅读“垃圾”代码

如何正确学习HDL代码:从描述的角度

模块名

模块输入、输出信号

```
module DFF_SYNC_PRE (Data, Clk, Preset, Q);
```

```
input Data, Clk, Preset;
```

```
output Q;
```

```
parameter U_DLY = 1;
```

```
reg Q;
```

```
always @ (posedge Clk )
```

```
    if ( ~Preset)
```

```
        Q    <= #U_DLY 1'b1 ;
```

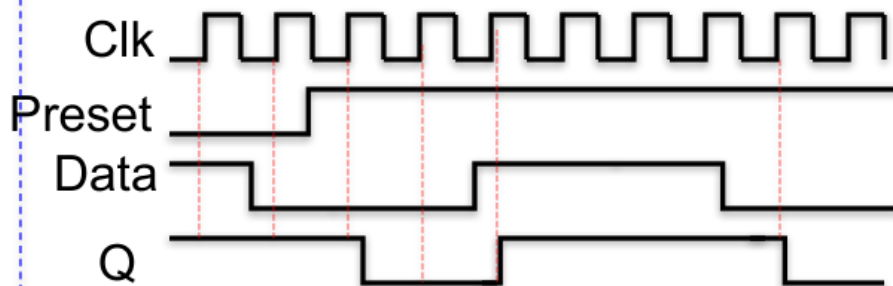
```
    else
```

```
        Q    <= #U_DLY Data ;
```

```
endmodule
```



DFF_SYNC_PRE



这段代码描述的是什么电路？

如何正确阅读HDL代码

```
module MUX(C,D,E,F,S,Mux_out);
```

```
input      C,D,E,F ;
```

```
input  [1:0]  S ;
```

```
output      Mux_out ;
```

```
reg Mux_out ;
```

```
always@(C or D or E or F or S)
```

```
begin
```

```
  case (S)
```

```
    2'b00 : Mux_out = C ;
```

```
    2'b01 : Mux_out = D ;
```

```
    2'b10 : Mux_out = E ;
```

```
    default : Mux_out = F ;
```

```
  endcase
```

```
end
```

这段代码描述的是什么电路？

多路复用器！

S为选择信号输入端

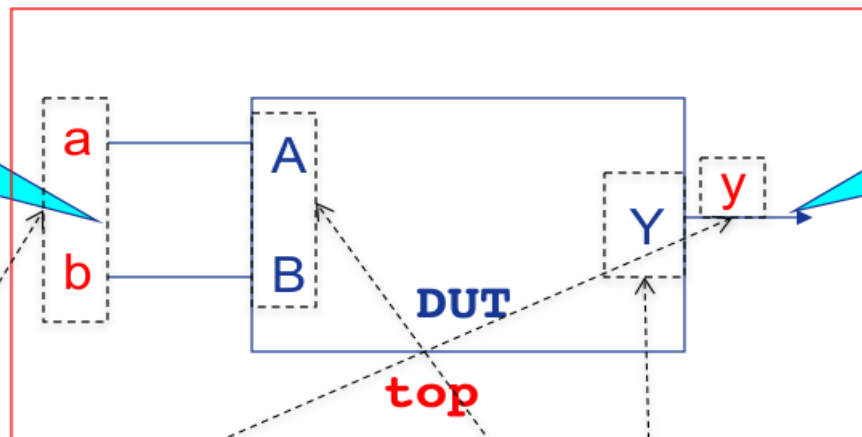
数据的输入输出均为**1**比特

Verilog HDL关键语法

输入端口可以由
net/register驱动
，但输入端口只能是
reg

双向端口输入/输出
只能是**net**类型

输出端口可以是
net/register类型
，输出端口只能驱动
net



```
module top;
  wire y;
  reg a, b;
  DUT u1 (y, a, b) ;
  initial begin
    a = 0; b = 0;
    #5 a = 1;
  end
endmodule
```

在过程块中只能
给**register**类型
赋值

```
module DUT (Y, A, B);
  output Y;
  input A, B;
  wire Y, A, B;
  and (Y, A, B) ;
endmodule
```

若Y, A, B说明
为**reg**则会产生
错误。

Verilog HDL关键语法

运算符

运算符分类	包含运算符
算术运算符	+、-、*、/、%
按位运算符	&、 、~、^、~^、
缩位运算符	&、 、~、^、~^、~&、~
逻辑运算符	&&、 、!
关系运算符	<=、>=、>、<
相等运算符	==、!=、!==(、===
移位运算符	<<、>>
拼接运算符	{ }
条件运算符	? :

形式上与C语言类似

实质上与电路对应!

Verilog HDL关键词法

◆ 算术操作符

常用的算术操作符：

+（加）、-（减）、*（乘）、/（除）、%（求余）。

其中%是求余操作符，在两个整数相除的基础上，取出其余数。例如，5 % 6的值为5；13 % 5的值是3。

◆ 逻辑操作符

逻辑操作符包括：

&&（逻辑与）、||（逻辑或）、!（逻辑非）

Verilog HDL关键词法

◆ 位运算

位运算是将两个操作数按对应位进行逻辑操作。位运算操作符包括：

~（按位取反）、&（按位与）、|（按位或）、^（按位异或）、^~或~^（按位同或）。

在进行位运算时，当两个操作数的位宽不同时，计算机会自动将两个操作数按右端对齐，位数少的操作数会在高位用0补齐。

◆ 关系操作符

<（小于）、<=（小于等于）、>（大于）、>=（大于等于）。其中，<=也是赋值运算的赋值符号。

关系运算的结果是1位逻辑值。在进行关系运算时，如果关系是真，则计算结果为1；如果关系是假，则计算结果为0；如果某个操作数的值不定，则计算结果不定（未知），表示结果是模糊的。

Verilog HDL关键词法

◆ 缩减操作符

缩减操作符包括：

& (与)、~& (与非)、| (或)、~| (或非)、
^ (异或)、^~或~^ (同或)。

缩减操作运算法则与逻辑运算操作相同，但操作的运算对象只有一个。在进行缩减操作运算时，对操作数进行与、与非、或、或非、异或、同或等缩减操作运算，运算结果有1位1或0。

例如，设 $A = 8' \text{ b}11010001$ ，则 $\& A = 0$ （在与缩减运算中，只有A中的数字全为1时，结果才为1）；

$|A = 1$ （在或缩减运算中，只有A中的数字全为0时，结果才为0）。

Verilog HDL 关键语法

◆ 转移操作符

转移操作符包括：

$\gg$ (右移)、 $\ll$ (左移)

操作数 $\gg n$; //将操作数的内容右移n位，同时从左边开始用0来填补移出的位数。

操作数 $\ll n$; //将操作数的内容左移n位，同时从右边开始用0来填补移出的位数。

例如，设 $A = 8' \text{ b11010001}$ ，则 $A \gg 4$ 的结果是 $A = 8' \text{ b00001101}$ ；而 $A \ll 4$ 的结果是 $A = 8' \text{ b00010000}$ 。

◆ 条件操作符

条件操作符为： $?:$

条件操作符的操作数有3个，其使用格式为

操作数 = 条件 ? 表达式1 : 表达式2;

即当条件为真（条件结果值为1）时，操作数 = 表达式1；
为假（条件结果值为0）时，操作数 = 表达式2。

Verilog HDL关键语法

◆ 位拼接操作符

拼接操作符为： **{ }**

拼接操作符的使用格式：

{操作数1的某些位, 操作数2的某些位, ..., 操作数n的某些位} ;

作用：即将操作数1的某些位与操作数2的某些位与...与操作数n的某些位拼接在一起。

例如，将1位全加器进位cont与和sum拼接在一起使用，它们的结果由两个加数ina、inb及低位进位cin相加决定的表达式为：

{cont, sum} = ina+inb+cin;

拼接操作符为：**N{ }**，表示复制N个封闭得字符串

例如：**{4{2' b01}}**的结果是：**8' b01010101**

Verilog HDL关键语法

◆ 赋值语句

(1) 门基元赋值语句

格式:

基本逻辑门关键字 (门输出, 门输入1, 门输入2, ..., 门输入n) ;

基本逻辑门关键字是Verilog HDL预定义的逻辑门, 包括and、or、not、xor、nand、nor等; 圆括弧中内容是被描述门的输出和输入信号。

nand (y, a, b, c, d) 和 $y = \sim (a \& b \& c \& d)$ 等效

(2) 连续赋值语句

格式:

assign 赋值变量 = 表达式;

Verilog HDL关键语法

(3) 过程赋值语句

过程赋值语句出现在initial和always块语句中，赋值符号是“=”，格式：

赋值变量 = 表达式;

在过程赋值语句中，赋值号“=”左边的赋值变量必须是reg（寄存器）型变量，其值在该语句结束即可得到。

如果一个块语句中包含若干条过程赋值语句，那么这些过程赋值语句是按照语句编写的顺序由上至下一条一条地执行，前面的语句没有完成，后面的语句就不能执行，就象被阻塞了一样。因此，过程赋值语句也称为阻塞赋值语句。

Verilog HDL关键语法

(4). 非阻塞赋值语句

非阻塞赋值语句也是出现在initial和always块语句中，赋值符号是“<=”，格式：

赋值变量 <= 表达式;

在非阻塞赋值语句中，赋值号“<=”左边的赋值变量也必须是reg型变量，其值不同于在过程赋值语句那样，语句结束时即刻得到，而在该块语句结束才可得到。

使用原则：

◆ 凡是在组合电路中被赋值的变量，必须是阻塞赋值。

always @ (*)

◆ 凡是在时序电路中被赋值的变量，必须是非阻塞赋值。

always @ (posedge clk or negedge rst_n)

Verilog HDL关键词法

◆ 条件语句

1. if-else语句

```
if (表达式)
    begin
        语句;
    end
else
    begin
        语句;
    end
```

```
if (表达式1)
    语句;
else if (表达式2)
    语句;
else if (表达式3)
    语句;
else
    语句;
```

Verilog HDL关键语法

2. case语句

case语句是一种多分支的条件语句，完整的case语句的格式为：

case (表达式)

分支1 : begin 语句1; end

分支2 : begin 语句2; end

...

分支n : begin 语句n; end

default : begin 语句n+1; end

endcase

case语句还有两种变体语句形式，即casez和casex语句。casez和casex语句与case语句的格式完全相同，它们的区别是：在casez语句中，如果分支表达式某些位的值为高阻z，那么对这些位的比较就不予以考虑，只关注其他位的比较结果。在casex语句中，把不予以考虑的位扩展到未知x，即不考虑值为高阻z和未知x的那些位，只关注其他位的比较结果。

Verilog HDL关键词法

◆ 循环语句

循环语句包含for语句、repeat语句、while语句和forever语句4种。

1. for语句

```
for (循环指针 = 初值; 循环指针 < 终值; 循环指针 = 循环指针 + 步长值)  
begin  
    语句;  
end
```

2. repeat语句

```
repeat (循环次数表达式)  
begin  
    语句;  
end
```

Verilog HDL关键语法

3、while 语句

while (循环执行条件表达式)

begin

 重复执行语句;

 修改循环条件语句;

end

while语句在执行时，首先判断循环执行条件表达式是否为真。

4、forever语句

forever

begin

 语句;

end

forever语句是一种无穷循环控制语句，常用来产生周期性的波形，作为仿真激励信号。

Verilog HDL关键语法

◆ always块语句-结构声明语句

在一个Verilog HDL模块 (module) 中, always块语句的使用次数是不受限制的, 块内的语句也是不断重复执行的。

always块语句的语法结构:

```
always @(敏感信号表达式)
```

```
begin
```

```
    // 过程赋值语句;
```

```
    // if语句, case语句;
```

```
    // for语句, while语句, repeat语句;
```

```
    // task语句、function语句;
```

```
end
```

Verilog HDL关键语法

- ◆ 敏感信号表达式分两种类型：电平敏感型和边沿敏感型。
- ◆ 每个always过程一般只由一种类型的敏感信号来触发，而不能混合使用。敏感信号表达式应该列出影响块内取值的所有信号（一般指设计电路的输入信号），多个信号之间用“or”连接。
- ◆ 对于组合逻辑电路，一般采用电平触发；
- ◆ 对于时序逻辑电路，一般采用时钟边沿触发（posedge或negedge）。

例如： `always @( posedge clk)`表示事件由clk的上升沿触发；`always @( negedge clk)`表示事件由clk的下降沿触发。

Verilog HDL关键语法

◆ initial语句

initial语句的语法格式为:

```
initial  
begin  
    语句1;  
    语句2;  
    ...  
end
```

initial语句的使用次数也是不受限制的，但块内的语句仅执行一次，因此initial语句常用于仿真中的初始化。

◆ assign语句

用assign语句建模的方法很简单，只需要在“assign”后面再加一个表达式即可。

assign语句一般适合对组合逻辑进行赋值，称为连续赋值方式。

Verilog HDL关键词法

◆ 端口位置关联

模块例化时，严格按照模块定义的端口顺序来连接，不用标明原模块定义时规定的端口名。

已定义模块: `module fullAdder(a,b,cin,sum,cout)`

实例化: `fullAdder adder(A,B,CIN,SUM,COUT);`

◆ 端口名称关联

模块例化时，端口和相应的外部信号按照其名字进行连接，而不是按照位置，端口连接可以以任意顺序出现。采用“.”符号，标明原模块定义时规定的端口名。

已定义模块: `module fullAdder(a,b,cin,sum,cout)`

实例化: `fullAdder adder(.sum(SUM), .cout(COUT), .b(B), .a(A), .cin(CIN));`

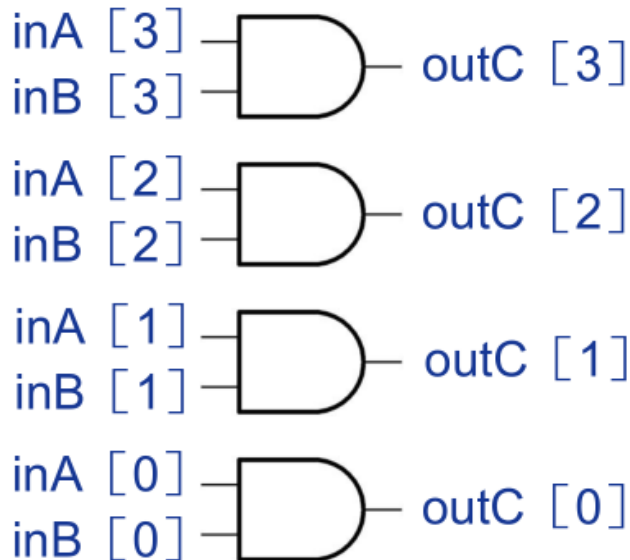
Verilog HDL关键语法

理解运算符语法和电路的对应关系

按位运算符

```
wire [3:0] inA ,inB;  
wire [3:0] outC;
```

```
assign outC = inA & inB;
```

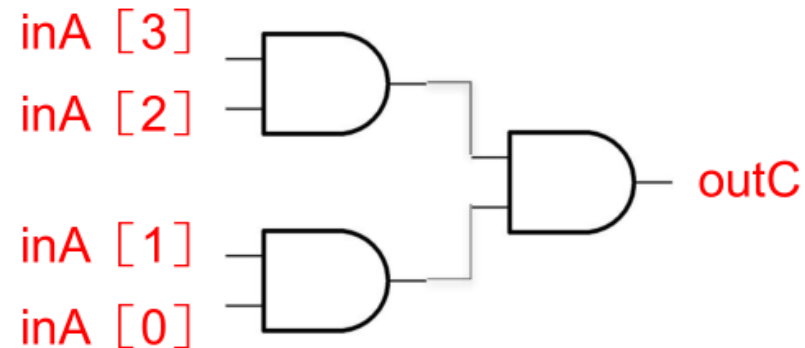


我举个栗子

缩位运算符

```
wire [3:0] inA ;  
wire      outC;
```

```
assign outC = & inA;
```



Verilog HDL关键词法

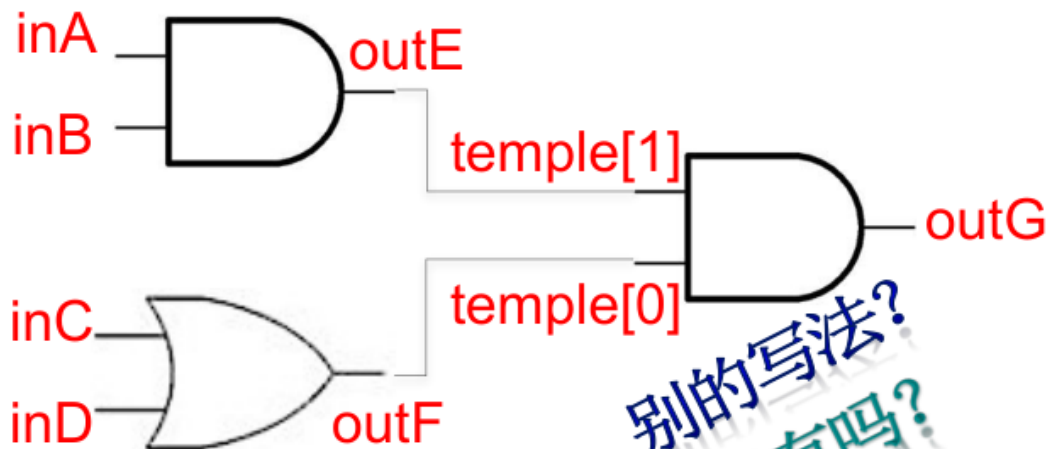


```
wire [1:0] temple;  
wire      inA,inB,inC,inD;  
wire      outE,outF,outG;
```

```
assign outE = inB & inA;  
assign outF = inC | inD;  
assign temple = {outE,outF} ;  
assign outG = &temple;
```

```
wire      inA,inB,inC,inD;  
wire      outE,outF,outG;
```

```
assign outE = inB & inA;  
assign outF = inC | inD;  
assign outG = outE & outF;
```



别的写法?
还有吗?

```
wire [1:0] temple;  
wire      inA,inB,inC,inD;  
wire      outE;
```

```
assign temple[1]= inB & inA;  
assign temple[0] = inC | inD;  
assign outE = &temple;
```

课堂练习1

- I. 构建一个4输入1输出的与非门电路，要求有仿真波形和板卡验证。
- II. 通过门级描述法，构建一个自定义门电路，该门电路需满足如下逻辑功能表达式： $Y=AB+AC+BC$
- III. 查阅相关技术资料，了解各逻辑运算的优先级

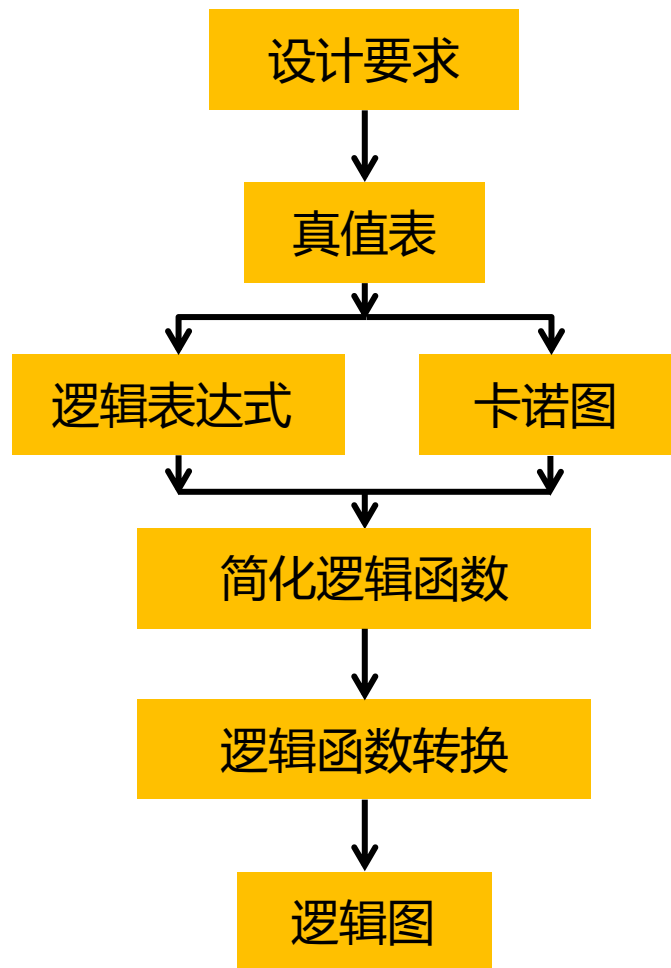


PART 02

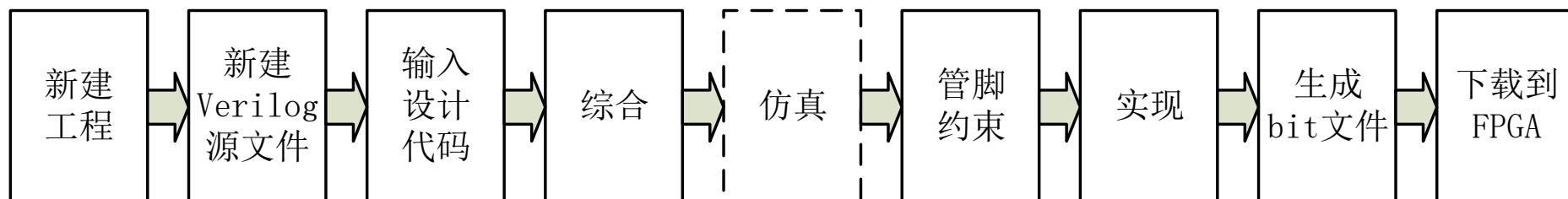
组合逻辑电路的设计

组合逻辑设计方法

组合逻辑电路的设计

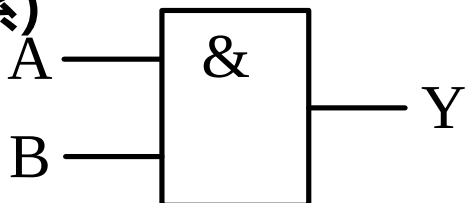


Verilog 建模方式



基于硬件描述语言的设计方法

(1) 结构化模型 (描述电路框架结构、
变量连接关系)



```
1 module and_gate_structure(A, B, Y);  
2   input A, B;  
3   output Y;  
4   and U1(Y, A, B);  
5 endmodule
```

(2) 数据流模型 (描述输入和输出
的逻辑关系)

$$Y = A \cdot B$$

```
1 module and_gate_dataflow(A, B, Y);  
2   input A, B;  
3   output Y;  
4   assign Y=A&B;  
5 endmodule
```

Verilog 建模方式

(3) 行为模型

输入		输出
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

```
1 module and_gate_behavioral(A, B, Y);  
2   input A, B;  
3   output reg Y;  
4   always @ (A or B)  
5   begin  
6       case ({A, B})  
7           2'b00: Y=1'b0;  
8           2'b01: Y=1'b0;  
9           2'b10: Y=1'b0;  
10          2'b11: Y=1'b1;  
11      endcase  
12  end  
13 endmodule
```



描述输入“行为”引起输出“行为”变化

基本的组合逻辑功能模块

Encoders (编码器)

Decoders (译码器)

Multiplexers (多路复用器)

Demultiplexers (多路分配器)

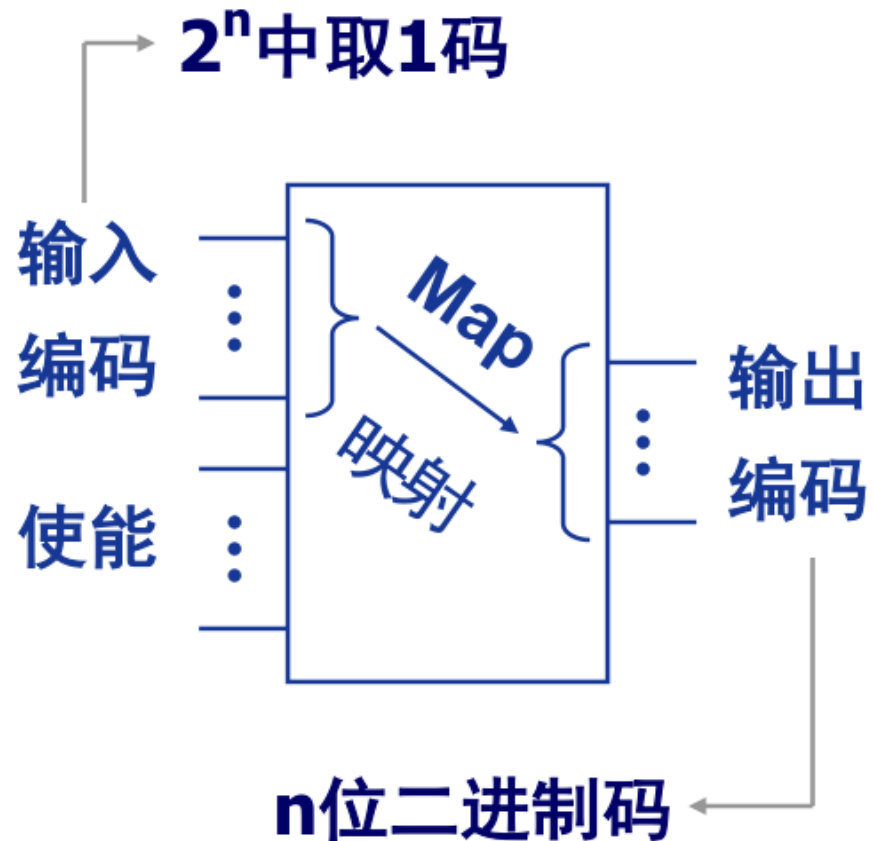
Parity Circuits (奇偶校验)

Comparators (比较器)

Adders (加法器)

编码器

- **编码器**是将信号（如比特流）或数据**进行编制、转换**为可用以通讯、传输和存储的信号形式的设备

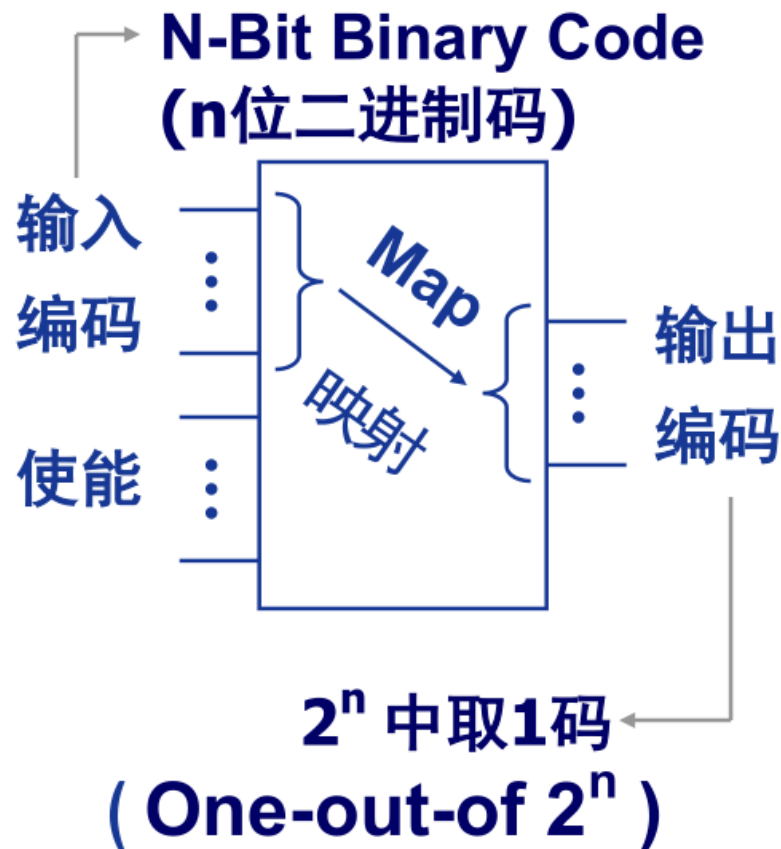


Encoder (编码器)

译码器

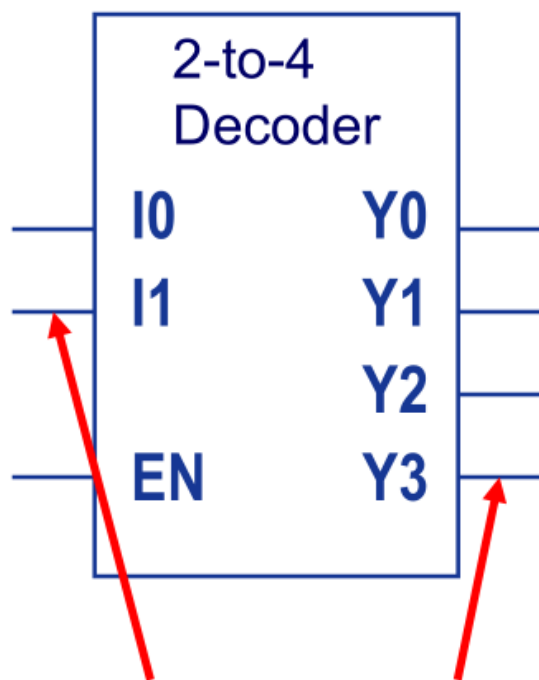
Decoder (译码器)

➤ **译码器**是一个多路输入、多路输出的组合逻辑电路，**它能把给定的一串二进制代码输入**译成一系列对应的输出高、低电平信号或一串新的代码，可用于数字显示、代码转换、数据分配、存储器寻址和组合控制信号等方面。



2-4二进制译码器

Binary Decoder (二进制译码器)



高位(MSB)在下面

Truth Table for a 2-to-4 Binary Decoder
(2-4二进制译码器真值表)

Inputs			Outputs			
EN	I1	I0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2-4二进制译码器

Truth Table for a 2-to-4 Binary Decoder

(2-4二进制译码器真值表)

$$Y0 = EN \cdot (I1' \cdot I0')$$

$$Y1 = EN \cdot (I1' \cdot I0)$$

$$Y2 = EN \cdot (I1 \cdot I0')$$

$$Y3 = EN \cdot (I1 \cdot I0)$$

$$Yi = EN \cdot m_i$$

Inputs			Outputs			
EN	I1	I0	Y3	Y2	Y1	Y0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2-4二进制译码器

$$Y_0 = EN \cdot (I_1' \cdot I_0')$$

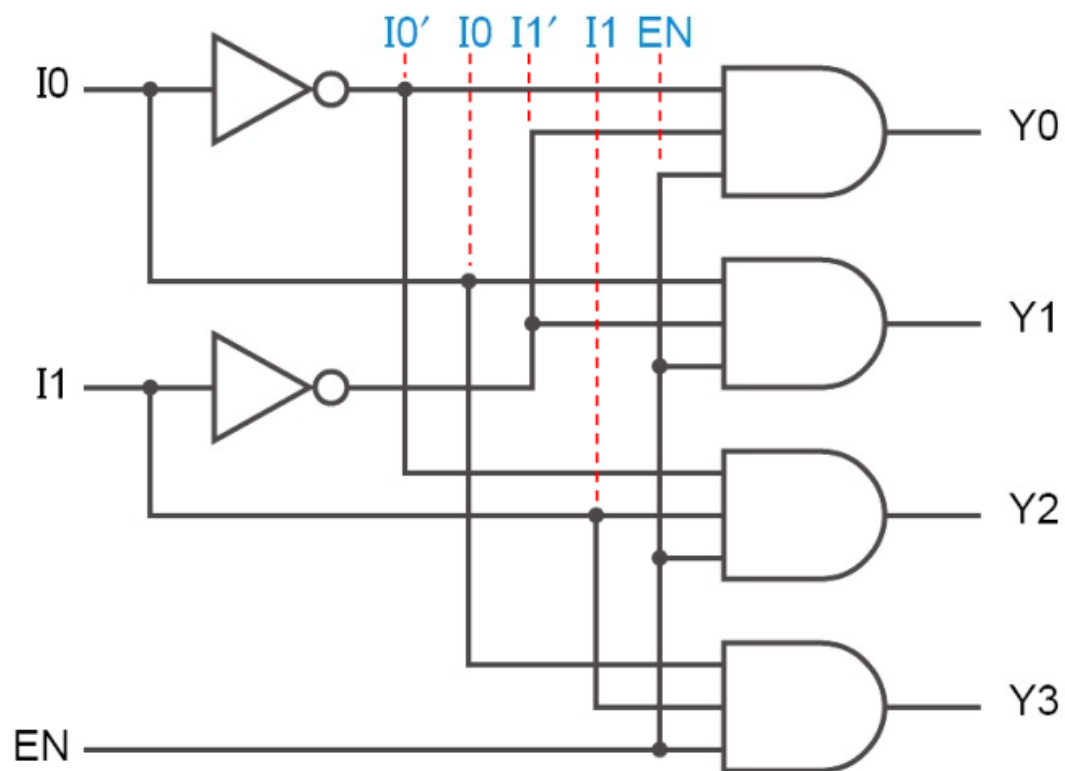
$$Y_1 = EN \cdot (I_1' \cdot I_0)$$

$$Y_2 = EN \cdot (I_1 \cdot I_0')$$

$$Y_3 = EN \cdot (I_1 \cdot I_0)$$

$$Y_i = EN \cdot m_i$$

逻辑表达式



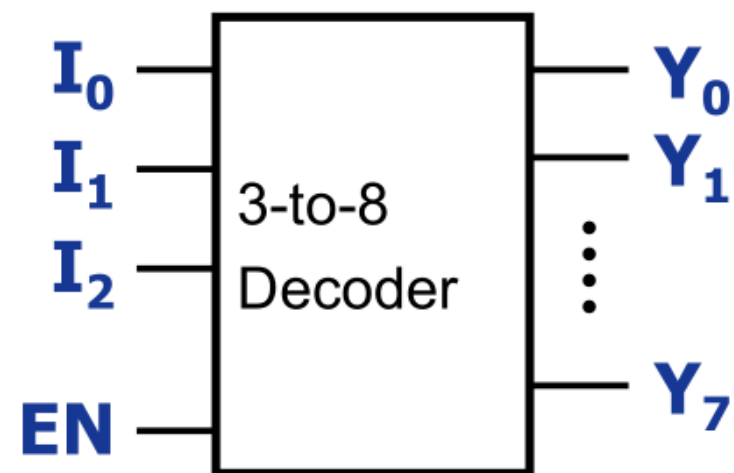
RTL逻辑电路图

3-8二进制译码器

Truth Table for a 3-to-8 Binary Decoder

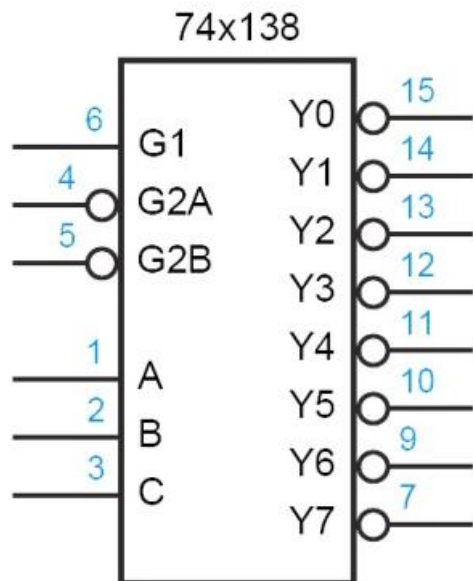
(3-8二进制译码器真值表)

$$Y_i = EN \cdot m_i$$

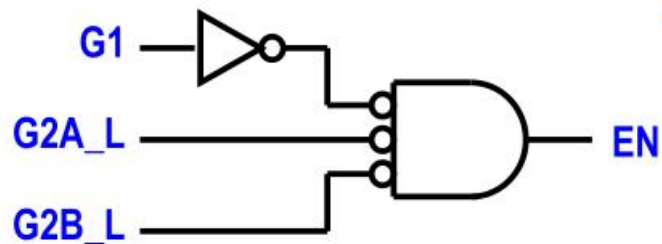


I_2	I_1	I_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	0	1
0	1	0	1	1	1	1	1	0	1	1
0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	1	1	0	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1
1	1	0	1	0	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1

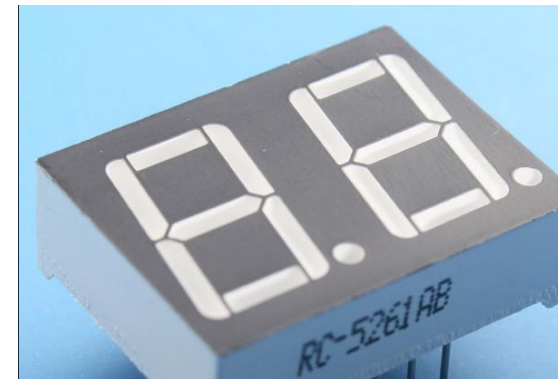
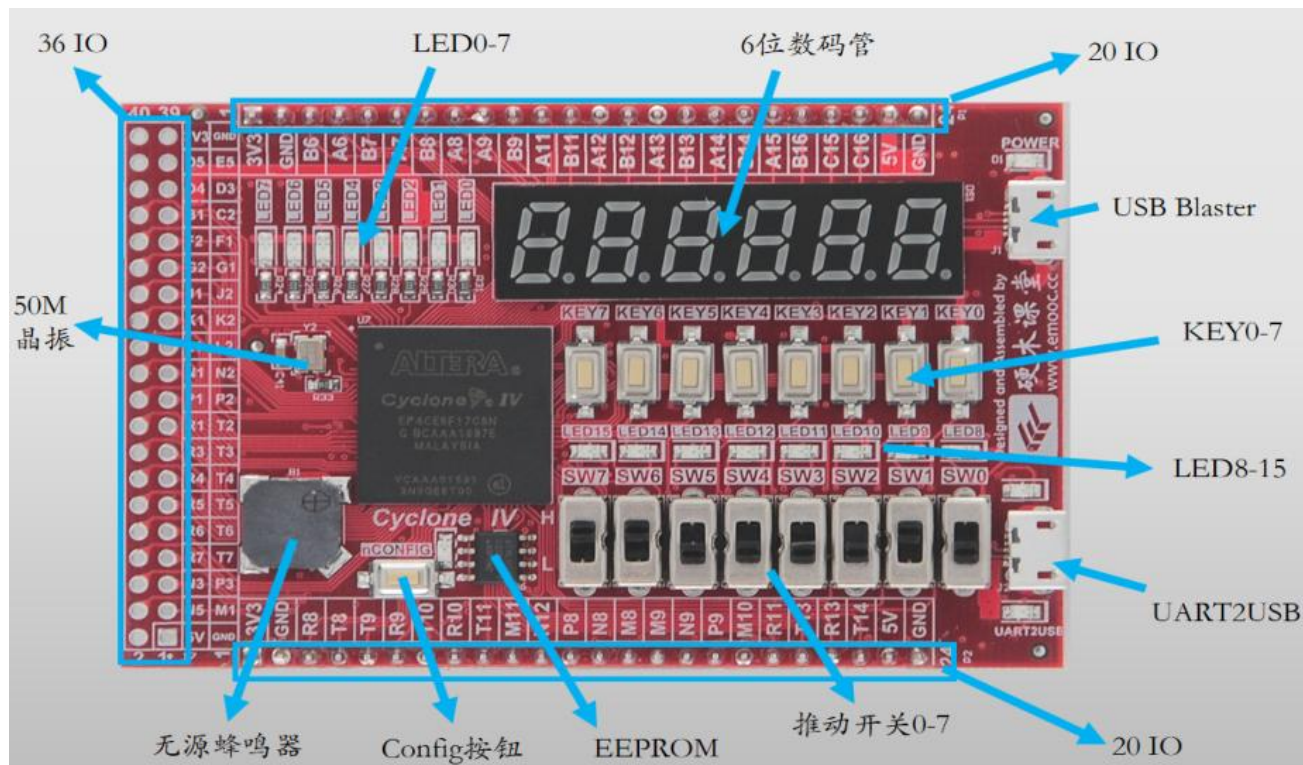
74LS138芯片引脚和功能



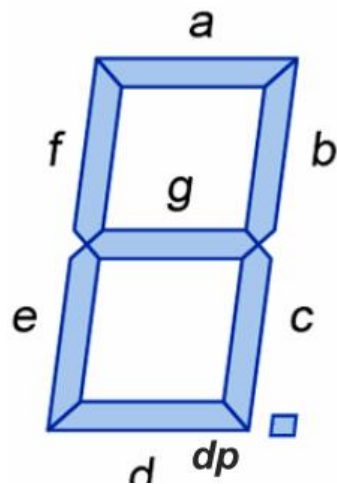
Inputs						Outputs							
G1	G2A_L	G2B_L	C	B	A	Y7_L	Y6_L	Y5_L	Y4_L	Y3_L	Y2_L	Y1_L	Y0_L
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1



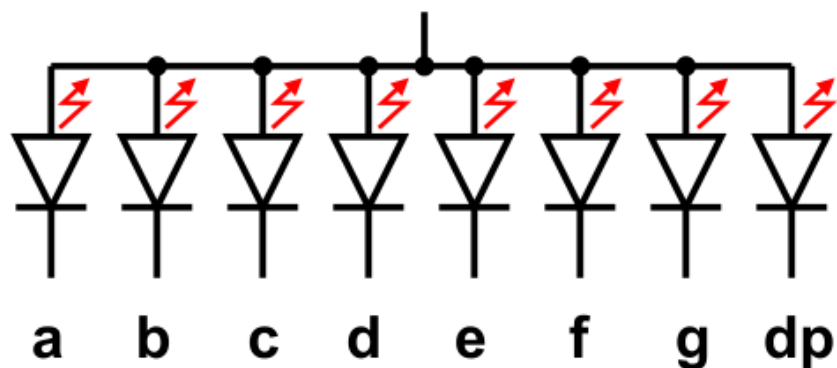
七段数码管



七段数码管译码器

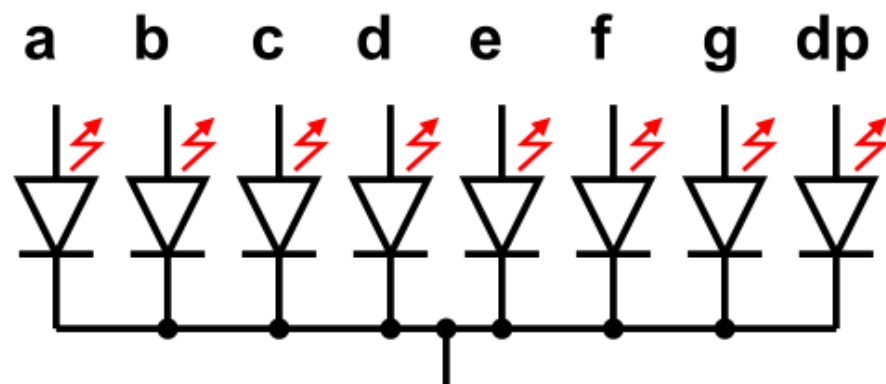


公共阳极



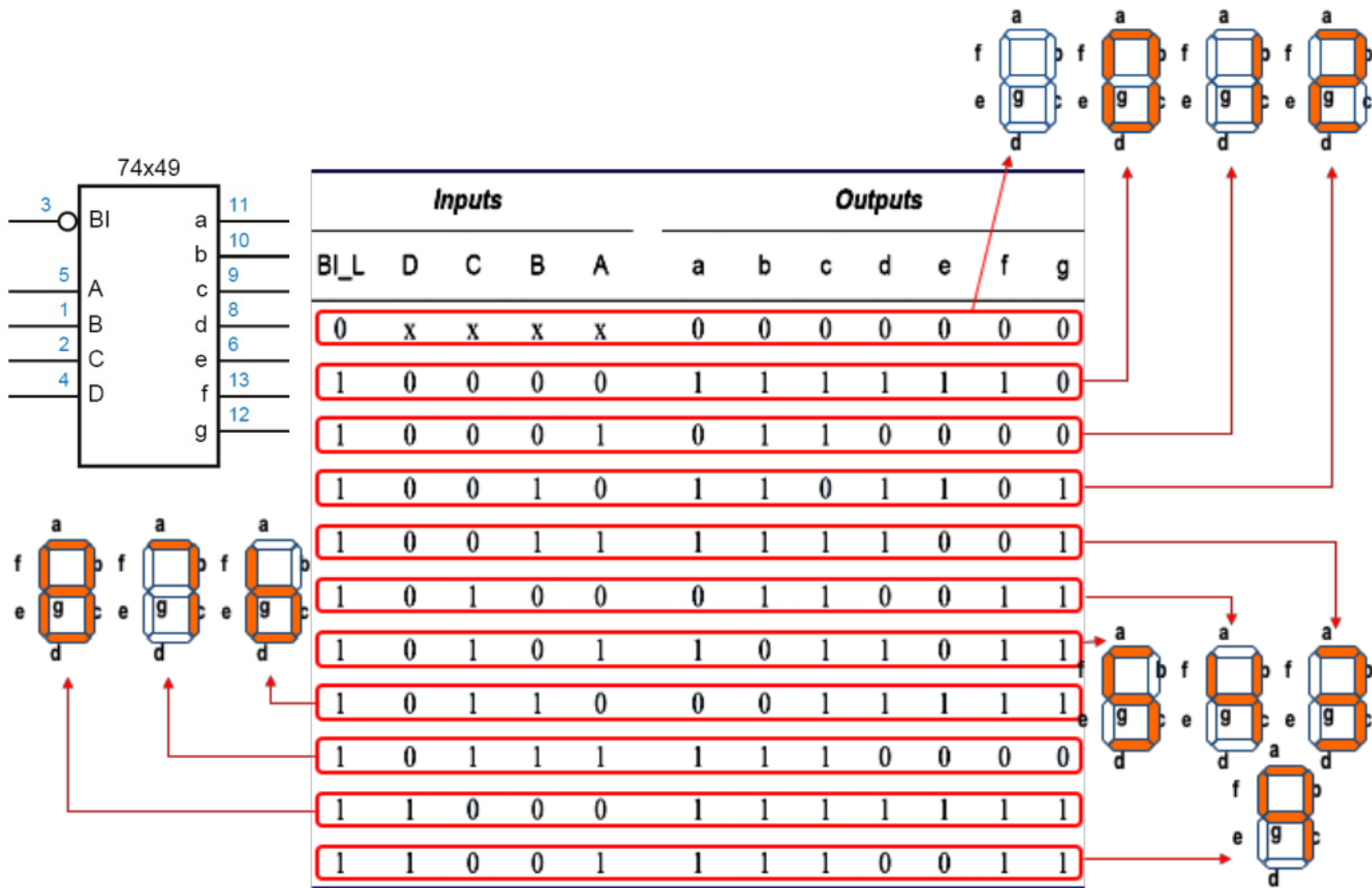
低有效

高有效



公共阴极

七段数码管译码器

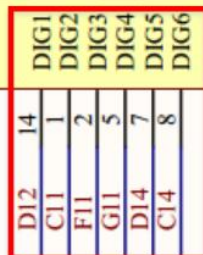
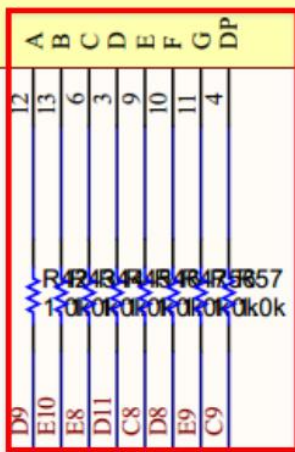
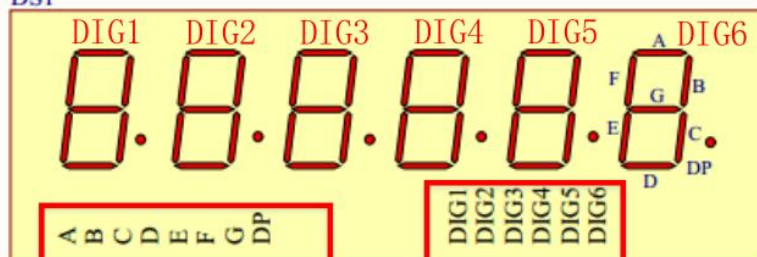


七段数码管译码器

六位数码管

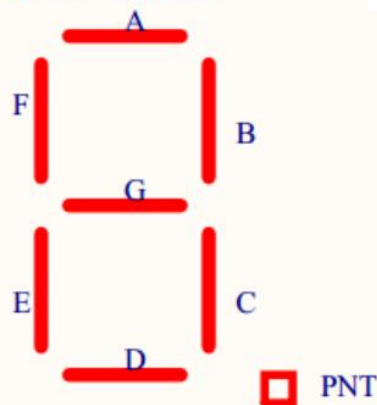
S03061AH-002

DS1



六个数码管的片选信号
引脚DigCS1-DigCS6

七段数码管的引脚
Digitron_Out[0:7]



动态显示数码管

多个数码管共用数据端

通过选择控制数码管轮流点亮



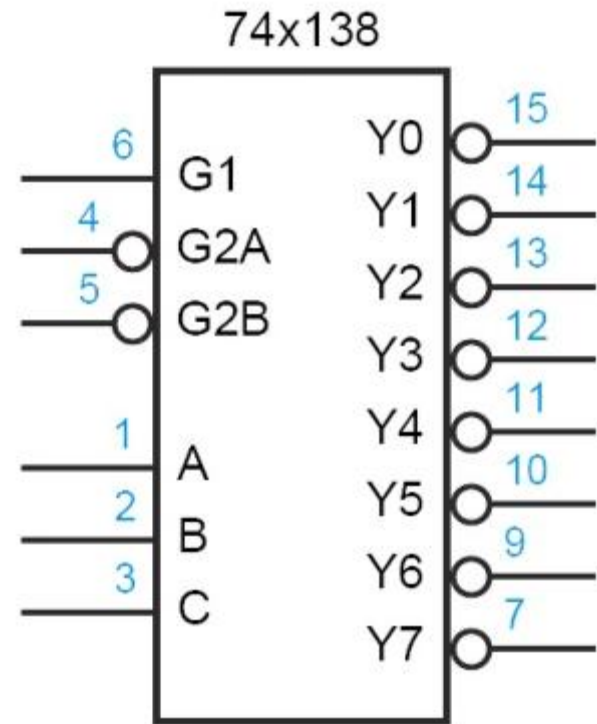
PART 03

用硬件描述语言描述芯片功能

用硬件描述语言描述74LS138

```
module msi74x138(G1,G2A_L, G2B_L, C, B, A, Y_L);  
  input G1, G2A_L, G2B_L;  
  input C, B, A;  
  output [7:0] Y_L;  
  reg [7:0] Y_L;
```

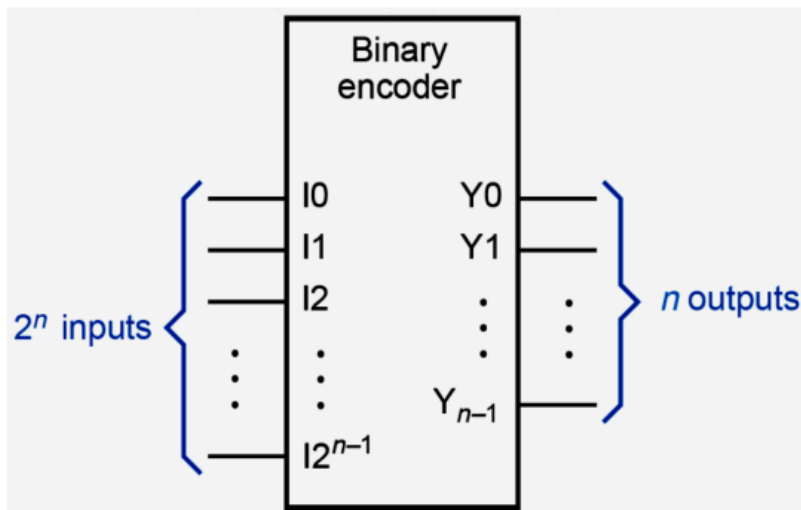
```
always @ (G1 or G2A_L or G2B_L or C or B or A)  
begin  
  if(G1 & ~G2A_L & ~G2B_L)  
    case({C,B,A})  
      3'b000: Y_L=8'b11111110;  
      3'b001: Y_L=8'b11111101;  
      3'b010: Y_L=8'b11111011;  
      3'b011: Y_L=8'b11110111;  
      3'b100: Y_L=8'b11101111;  
      3'b101: Y_L=8'b11011111;  
      3'b110: Y_L=8'b10111111;  
      3'b111: Y_L=8'b01111111;  
      default: Y_L=8'b11111111;  
    endcase  
  else  
    Y_L=8'b11111111;  
  end  
end  
endmodule
```



74LS138芯片引脚图

编码器

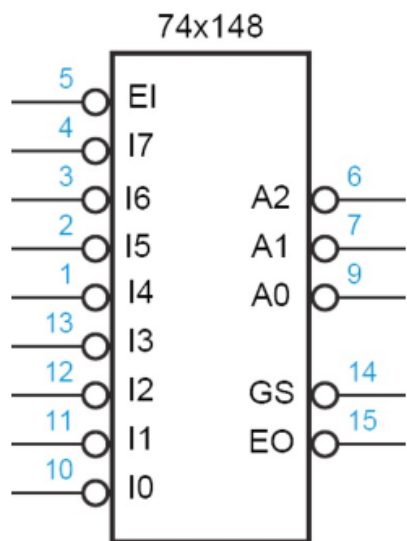
Truth Table for a 8-to-3 Encoder
(3位二进制编码器的真值表)



I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	A_2	A_1	A_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

优先编码器

优先编码器真值表

[illegible]

Verilog 描述优先编码器

```

module msi74x148(EI_L, I_L, A_L, GS_L, EO_L);
  input EI_L;
  input [7:0] I_L;
  output GS_L, EO_L;
  output [2:0] A_L;
  reg GS_L, EO_L;
  reg [2:0] A_L;

```

```

always @ (EI_L or I_L)
  begin
    if(~EI_L)
      casex(I7_L)
        8'b0xxxxxxx:
          begin
            A_L=3'b000;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        8'b10xxxxxx:
          begin
            A_L=3'b001;
            GS_L=1'b0;
            EO_L=1'b1;
          end

```

```

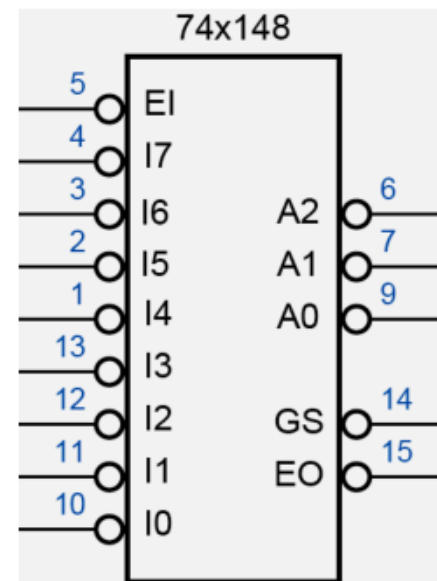
        8'b110xxxxx:
          begin
            A_L=3'b010;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        8'b1110xxxx:
          begin
            A_L=3'b011;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        8'b11110xxx:
          begin
            A_L=3'b100;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        8'b111110xx:
          begin
            A_L=3'b101;
            GS_L=1'b0;
            EO_L=1'b1;
          end

```

```

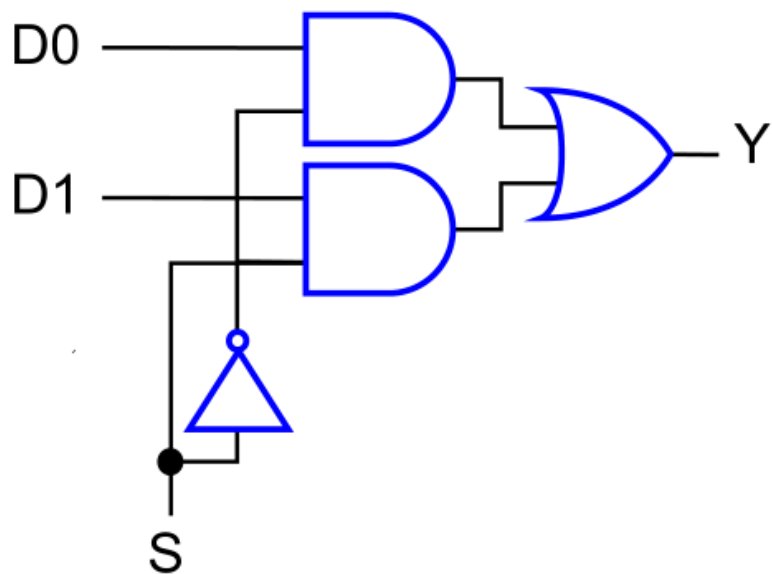
        8'b1111110x:
          begin
            A_L=3'b110;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        8'b11111110:
          begin
            A_L=3'b111;
            GS_L=1'b0;
            EO_L=1'b1;
          end
        default:
          begin
            A_L=3'b111;
            GS_L=1'b1;
            EO_L=1'b0;
          end
      endcase
    else
      begin
        A_L=3'b111;
        GS_L=1'b1;
        EO_L=1'b1;
      end
    end
  end
endmodule

```

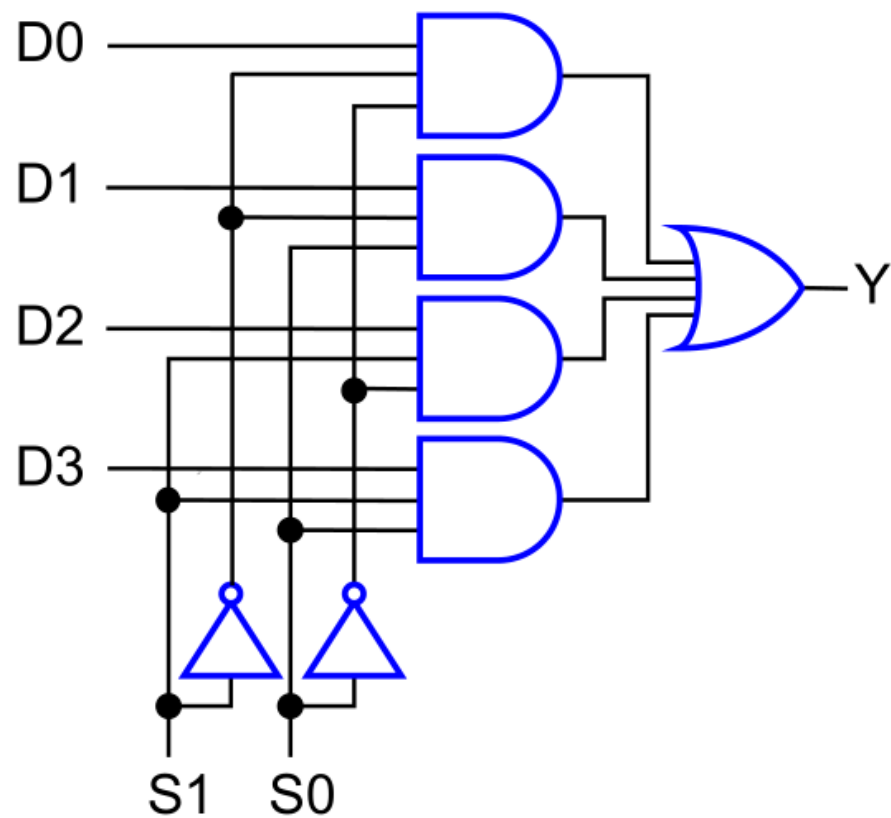


多路复用器

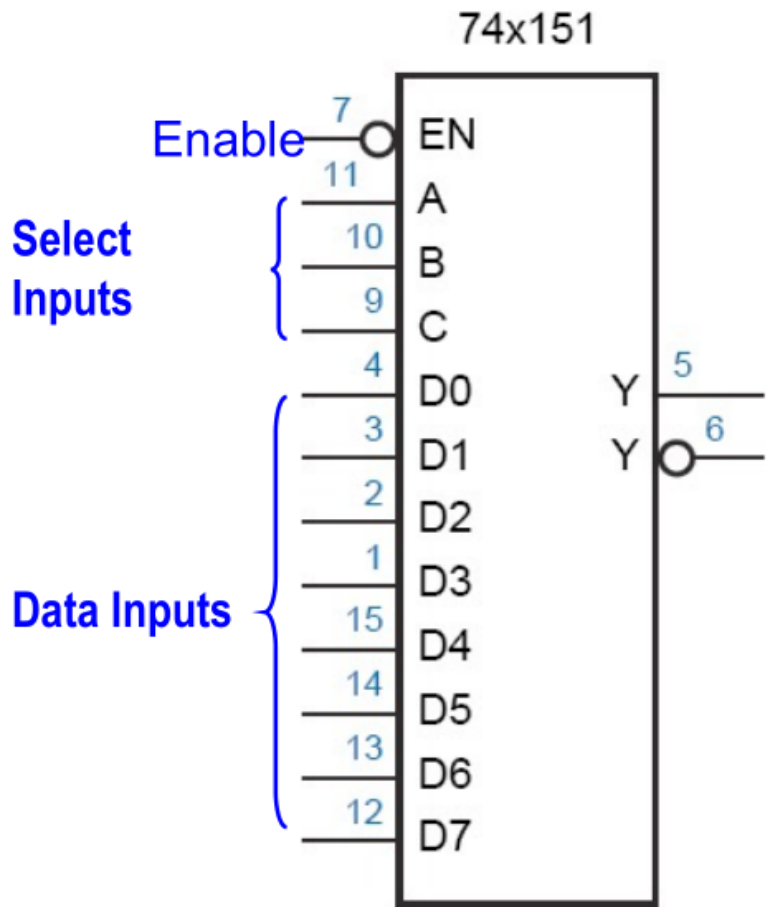
2x1 MUX



4x1 MUX



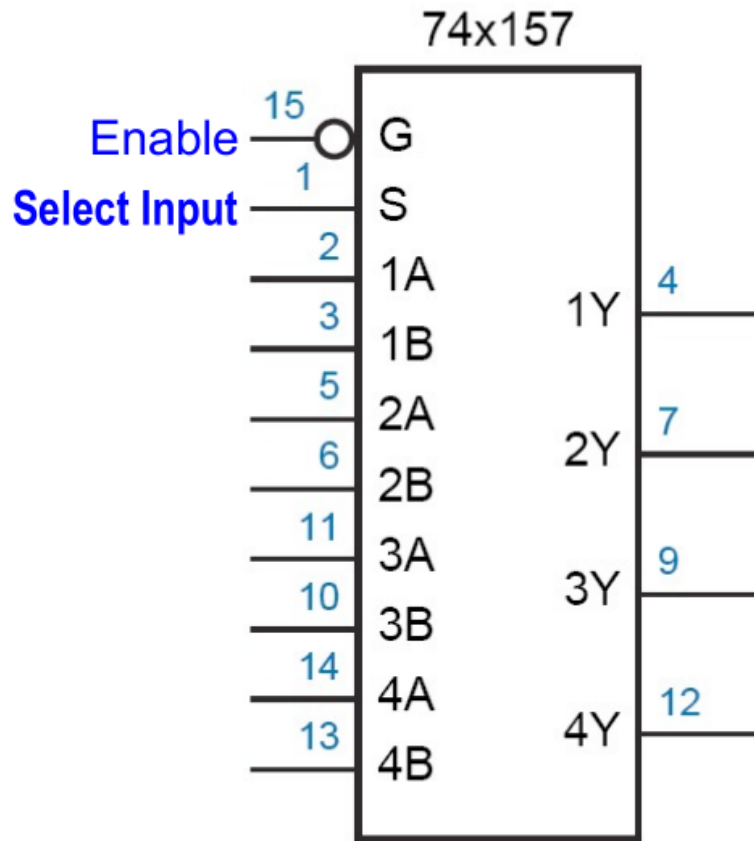
多路复用器—8选1 (1位)



Truth Table

EN_L	C	B	A	Y	Y_L
1	X	X	X	0	1
0	0	0	0	D0	D0'
0	0	0	1	D1	D1'
0	0	1	0	D2	D2'
0	0	1	1	D3	D3'
0	1	0	0	D4	D4'
0	1	0	1	D5	D5'
0	1	1	0	D6	D6'
0	1	1	1	D7	D7'

多路复用器—2选1 (4位)



Truth Table

G_L	S	1Y	2Y	3Y	4Y
1	X	0	0	0	0
0	0	1A	2A	3A	4A
0	1	1B	2B	3B	4B

Verilog语言描述多路复用器8选1

```
module msi74x151(EN_L, A, D, Y, Y_L);
```

```
    input EN_L;
```

```
    input [2:0] A;
```

```
    input [7:0] D;
```

```
    output Y, Y_L;
```

```
    reg Y;
```

```
    always @ (EN_L or A or D)
```

```
    begin
```

```
        if(~EN_L)
```

```
            case(A)
```

```
                3'b000: Y=D[0];
```

```
                3'b001: Y=D[1];
```

```
                3'b010: Y=D[2];
```

```
                3'b011: Y=D[3];
```

```
                3'b100: Y=D[4];
```

```
                3'b101: Y=D[5];
```

```
                3'b110: Y=D[6];
```

```
                3'b111: Y=D[7];
```

```
            endcase
```

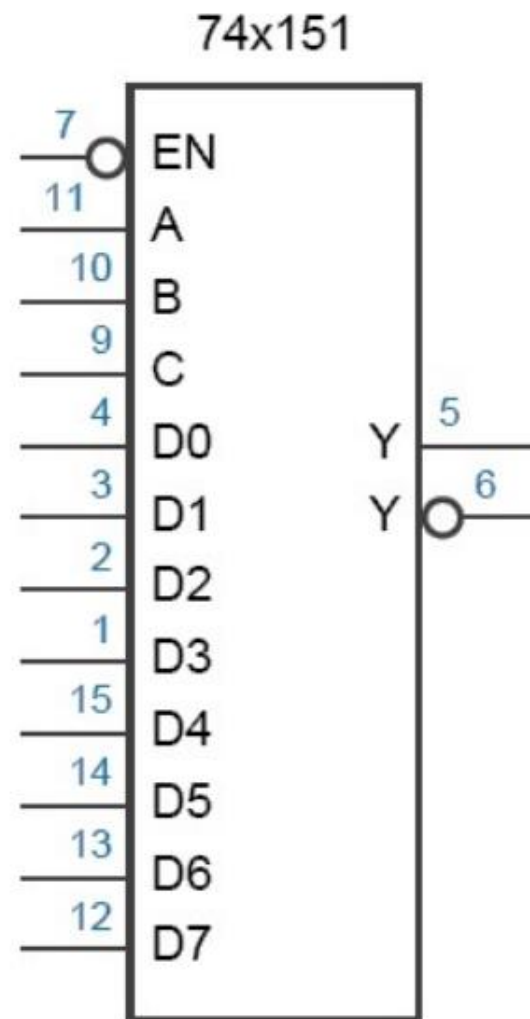
```
        else
```

```
            Y=1'b0;
```

```
    end
```

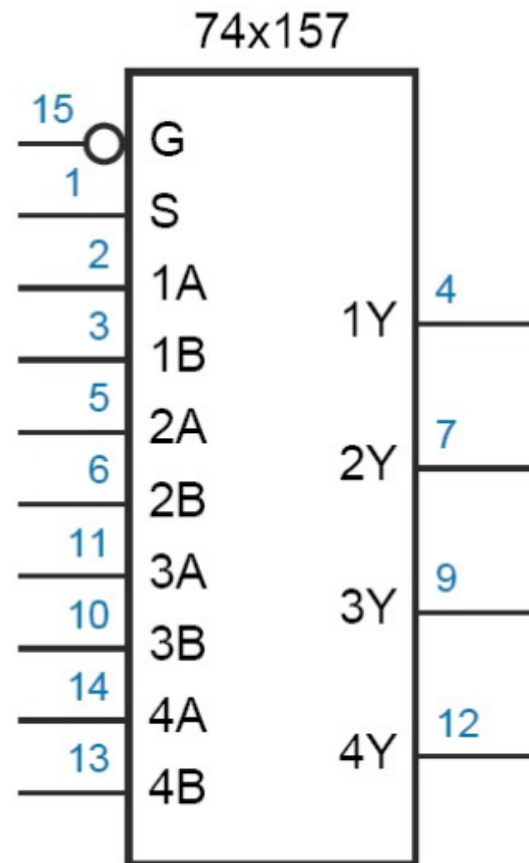
```
    assign Y_L=~Y;
```

```
endmodule
```



Verilog语言描述多路复用器2选1

```
module msi74x157(G_L, S, A, B, Y);  
  input  G_L, S;  
  input [3:0] A, B;  
  output [3:0] Y;  
  reg [3:0] Y;  
  
  always @ (G_L or S or A or B)  
  begin  
    if(~G_L)  
      case(S)  
        1'b0: Y=A;  
        1'b1: Y=B;  
      endcase  
    else  
      Y=4'h0;  
    end  
  
  endmodule
```





PART 04

课堂练习

课堂练习2

❖1、单数码管显示

- 用4位拨码开关（**SW3-SW0**）作为输入
- 用**DIG1**显示，16进制显示

❖2、输入请求响应显示

- 使用8位拨码开关（**SW7-SW0**）作为输入
- 其中**SW7**优先级最高，**SW0**优先级最低
- 输入为1时有效，输入为0时无效
- 数码管显示优先级最高的有效位，例如此时输入为**00110110**时，数码管显示为**5**
- 当输入为全0时则输出时数码管不亮

课堂练习2

❖ 3、数码管动态显示

- 用4位拨码开关（**SW3-SW0**）作为数据输入
- 用3位拨码开关（**SW7-SW5**）作为控制输入
- 用控制输入决定显示的位置
- 例如**SW7-SW5**输入为**010**时，数码管**DIG2**显示数据，**SW7-SW5** 为**000**和**111**时不显示

❖ 4、两位数动态显示

- 使用8位拨码开关（**SW7-SW0**）作为输入
- 其中**SW7-SW4**表示1个数，**SW3-SW0**表示另外一个数
- 使用按键**Key0**作为控制，**Key=0**时**DIG1**显示**SW3-SW0**的数，**KEY=1**时显示**DIG2**显示**SW7-SW5**的数